

```

# STEP 1: Product Catalog
def displayCatalogue(catalog: list[tuple]):
    "Print the list of all products and prices"
    print("What would you like to buy?")
    for name, price in catalog:
        print(f"-{name}:${price:.2f}")

def getProductPrice(catalog: list[tuple], productName: str):
    "Search for a product name and return its price"
    for name, price in catalog:
        if name.lower() == productName.lower():
            return price
    return None

# STEP 2: Cart Operations
def checkStock(stock: dict, product: str, qty: int) -> bool:
    """ Check if product exists in the stock
    If yes, the check if qty is sufficient"""
    if product in stock:
        if stock[product] >= qty:
            return True
        else:
            return False
    else:
        return False

def addToCart(cart: list[dict], stock: dict, catalog: list[tuple],
product: str, qty: int):
    """Check for validity and availability of stock and update
    Update the qty when added to cart"""
    price = getProductPrice(catalog, product)

    if price is None:
        print("Product not found")
    else:
        if checkStock(stock, product, qty):
            cart.append({"product": product, "qty": qty})
            stock[product] -= qty
            print(f"{qty} x {product} added to cart")
        else:
            print("Not enough stock")

def removeFromCart(cart: list[dict], product: str):
    """Remove items from the cart using cart index"""
    for i in range(len(cart)):
        if cart[i]["product"].lower() == product.lower():
            cart.pop(i)
            print(f"Removed {product} from cart")
            return
    print("Item not in cart")

def viewCart(cart: list[dict]):
    """Display contents in the cart"""

```

```

print("\n Your Cart: ")
if not cart:
    print(" Cart is empty")
else:
    for item in cart:
        print(f"-{item['qty']} x {item['product']}")

# STEP 3: Checkout
def calculateTotal( cart: list[dict], catalog: list[tuple]) -> float:
    """Sum price * qty for all cart items"""
    total = 0.0
    for item in cart:
        for name, price in catalog:
            if name.lower() == item["product"].lower():
                total += price * item["qty"]
                break
    return total

def applyDiscount(total: float)-> float:
    """10% off if total > 2000 and 5% off if total > 1000"""
    if total > 2000:
        return total * 0.90
    elif total > 1000:
        return total * 0.95
    else:
        return total

def applyTax(total: float)->float:
    """8% sales tax"""
    return total * 1.08

def generateReceipt(cart: list[dict], catalog: list[tuple]):
    """Print receipt"""
    print("\n****Receipt****")
    subtotal = calculateTotal(cart, catalog)
    discounted = applyDiscount(subtotal)
    taxed = applyTax(discounted)
    for item in cart:
        price = getProductPrice(catalog, item["product"])
        details = price * item["qty"]
        print(f" {item["qty"]} x {item["product"]} @ ${price:.2f} =
${details:.2f}")
    print(f" Subtotal: ${subtotal: .2f}")
    print(f" Discount applied: ${discounted: .2f}")
    print(f" Total with tax: ${taxed: .2f}")

# STEP 4: Recursive Recommendation
def recommendProduct(catalog: list[tuple], cart: list[dict], index: int =
0):
    """Recursively return first catalog item not in cart"""
    if index >= len(catalog):
        return None
    product = catalog[index][0]
    if all(item["product"].lower() != product.lower() for item in cart):

```

```

        return product
    return recommendProduct(catalog, cart, index+1)

# STEP 5: Order History
def updateOrderHistory(orderHistory: list[tuple], cart: list[dict]):
    """Save order history as tuple"""
    snapshot = []
    for item in cart:
        # extract product name and quantity
        prod = item["product"]
        quantity = item["qty"]
        snapshot.append((prod, quantity))

    # Convert to tuple and append to history
    orderHistory.append(tuple(snapshot))

def summarizeOrders(orderHistory: list[tuple]):
    """Print past orders"""
    print("\n Order History:")
    if not orderHistory:
        print(" No History")
        return
    i = 1
    for order in orderHistory:
        itemsList = []
        for prod, quantity in order:
            itemsList.append(f"{quantity} x {prod}")
        items = ",".join(itemsList)
        print(f"{i}. {items}")
        i += 1

# STEP 6: Sales Analytics
def getTopProducts(salesData: dict):
    """Show top 3 selling products"""
    print("\n Top 3 products")
    top = sorted(salesData.items(), key=lambda x: x[1], reverse=True)[:3]
    for prod, count in top:
        print(f" {prod}: {count} sold")

def stringMatchScore(str1: str, str2: str) -> int:
    """Return count of matching chars at same positions."""
    score = 0
    for i in range(min(len(str1), len(str2))):
        if str1[i] == str2[i]:
            score += 1
    return score

def main():
    catalog = [("Bag", 75), ("Laptop", 450), ("Playstation", 699.99),
("Table", 999)]
    stock = {"Bag": 500, "Laptop": 50, "Playstation": 75, "Table": 30}
    cart=[]
    orderHistory=[]
    salesData={name: 0 for name,_ in catalog}

```

```

while True:
    print("****Smart Cart Menu****\n"\
          "1: View Catalog\n"\
          "2: Add to Cart\n"\
          "3: Remove from Cart\n"\
          "4: View Cart\n"\
          "5: Checkout\n"\
          "6: Order History\n"\
          "7: Top 3 products\n"\
          "8: Recommend Product\n"\
          "9: Exit")
    select = input("What would you like to do: ").strip()
    if select == '1':
        displayCatalogue(catalog)
    elif select == '2':
        prod = input("Enter product to be added: ")
        quantity = int(input("Enter quantity: "))
        addToCart(cart, stock, catalog, prod, quantity)
    elif select == '3':
        prod = input("Enter product to be removed: ")
        removeFromCart(cart, prod)
    elif select == '4':
        viewCart(cart)
    elif select == '5':
        generateReceipt(cart, catalog)
        for item in cart:
            salesData[item["product"]] += item["qty"]
        updateOrderHistory(orderHistory, cart)
        cart.clear()
    elif select == '6':
        summarizeOrders(orderHistory)
    elif select == '7':
        getTopProducts(salesData)
    elif select == '8':
        rec = recommendProduct(catalog, cart)
        print(f"\n Try: {rec}" if rec else "\n No new products to
recommend.")
    elif select == '9':
        print("Goodbye!")
        break
    else:
        print("Invalid entry")

if __name__=="__main__":
    main()

```